

Avaliação de Desempenho e Otimização de Código com Locust

Objetivo

Medir, relatar e otimizar o desempenho básico de uma API REST monolítica usando o Locust, comparando o comportamento do sistema antes e depois de uma refatoração de código.

O que medir

1. **Tempo médio de resposta (ms)** - quanto tempo, em média, cada requisição demorou.
2. **Tempo máximo de resposta (ms)** - o pior caso observado.
3. **Requisições por segundo (req/s)** - quantas requisições o sistema atende por segundo (*Vazão/Throughput*).
4. **Total de requisições** - volume atendido em cada cenário.
5. **Erros** - quantidade de respostas 4xx e 5xx e % de sucesso.

Aplicação-alvo

Vocês devem utilizar uma API REST simples (monolítica), desenvolvida na linguagem ou framework de preferência da dupla (ex: Node.js, Python/FastAPI, Java/Spring Boot).

Esta API deve rodar localmente e possuir **pelo menos um endpoint de leitura (GET) que realize uma operação propositalmente "pesada" ou mal estruturada**. Exemplos de cenários comuns que vocês podem simular:

- **Exemplo 1 (Banco de Dados):** Um endpoint que faz múltiplas consultas individuais ao banco de dados dentro de um laço de repetição, em vez de uma única consulta com **JOIN** ou paginação.
- **Exemplo 2 (Processamento/CPU):** Um endpoint que realiza uma busca linear ou processamento pesado de Strings/Arquivos de forma síncrona a cada requisição, que poderia ser otimizado usando algoritmos melhores, *caching* em memória, ou assincronismo.

Endpoints a exercitar (plano de teste)

No mundo real, os usuários não fazem todos a mesma coisa ao mesmo tempo. Para simular o comportamento de produção (o que chamamos de **Mix de Carga** ou *Workload Mix*), a ferramenta não vai testar apenas a rota lenta, mas sim distribuir os acessos entre **4 endpoints diferentes**. Configure as tarefas no Locust para seguir a distribuição abaixo, focando o maior volume de estresse justamente na rota onde você criou o gargalo de código:

- GET /api/recurso-lento — 40% dos acessos (*O endpoint com o gargalo que será otimizado por você*)
- GET /api/recurso-detalle/{id} — 30% dos acessos (*Uma consulta simples e indexada por um ID específico*)
- GET /api/status — 20% dos acessos (*Uma rota leve de verificação de integridade/healthcheck*)
- POST /api/recurso — 10% dos acessos (*Um cadastro simples de novos dados no sistema*)

Cargas Experimentais

O experimento será dividido em duas fases para analisar o impacto real da alteração do código:

- **Fase 1 (Baseline - Código Original):** Execução do teste contra a API em seu estado inicial lento com **50 usuários simultâneos por 5 minutos** (descarte o 1º minuto como aquecimento/*warm-up*).
- **Fase 2 (Otimizado - Código Refatorado):** Execução do teste contra a API após a correção do gargalo, mantendo exatamente a mesma carga de **50 usuários simultâneos por 5 minutos** (descarte o 1º minuto).

Faça 5 repetições de cada cenário e use a média dos resultados para mitigar oscilações locais do sistema operacional.

Procedimento (passo a passo)

1. Suba a API Monolítica em sua versão inicial (com o gargalo de código) e confirme que responde nos endpoints acima.
2. Popule o banco de dados ou a memória com dados mínimos para que as consultas retornem registros válidos.
3. Rode a **Fase 1 (baseline)** no Locust e salve os CSVs de resultados.
4. Refatore o código do endpoint lento aplicando técnicas de engenharia de software para eliminar o gargalo.
5. Rode a **Fase 2 (otimizado)** com a aplicação otimizada e extraia os novos CSVs.
6. Para cada cenário, anote: tempo médio, tempo máximo, req/s, total de requisições, erros e % de sucesso.
7. Compare os dois cenários em tabelas e gráficos simples.
8. Escreva as conclusões encontradas no experimento.

Entregáveis

1. **Repositório com:**
 - O locustfile.py (tarefas com o mix acima).
 - Código-fonte da aplicação no GitHub.
 - Pasta results/ com CSVs do Locust e uma planilha/tabela de resumo.
 - README.md com passos de reprodução (como clonar, instalar dependências e rodar o teste).
2. **Vídeo** mostrando o funcionamento do sistema e seu monitoramento durante o teste de carga.
3. **Relatório** (padrão IEEE no Overleaf) contendo:
 - Resumo (o que foi feito e principais números).
 - Descrição do sistema testado e a justificativa técnica do gargalo escolhido.
 - Como o teste foi feito (especificações da máquina local e cenários).
 - Resultados (tabelas e gráficos das métricas simples comparando Baseline vs Otimizado).
 - Conclusões (o impacto real da refatoração nas métricas de desempenho).
 - Limitações (o que não foi medido; interferências do hardware host).

Só postagem, não terá apresentação presencial.